



UNIVERSIDADE DO ESTADO DO RIO GRANDE DO NORTE
CURSO DE SISTEMAS PARA INTERNET – EAD

ERIOMAR KALIEUDES MORAIS E SILVA – POLO APODI
THALES DE FREITAS FILGUEIRA – POLO APODI

Conversão de um banco de dados relacional para um não relacional

APODI

2026

Introdução

Ao desenvolver um projeto de software a escolha do banco de dados entre relacional e NoSQL é fundamental, pois isso irá impactar o desempenho e a flexibilidade do software.

Os bancos de dados relacionais, como SQL Server, MySQL, PostgreSQL etc., obedecem a uma estrutura de tabelas e colunas que são interconectadas por chaves primárias (primary key) e chave estrangeira (foreign key), e utilizam a linguagem SQL para fazer consultas, inserções, atualizações e exclusão de dados. Esse tipo de banco se torna ideal para sistemas que exigem consistência, organização e segurança.

Em contrapartida, também nos disponibilizamos dos bancos NoSQL (não relacional), que surgiram como alternativa para as limitações que os bancos relacionais tem diante das altas demandas da era digital, como por exemplo o MongoDB, Redis, Cassandra etc. Esse modelo de banco oferece flexibilidade pois não exigem um esquema fixo assim como os relacionais, e conseguem lidar com grandes volumes de dados e escalar horizontalmente.

O presente projeto tem como objetivo realizar a conversão de um banco de dados relacional de uma biblioteca acadêmica para um banco de dados orientado a documentos utilizando MongoDB. Para garantir as boas práticas de desenvolvimento, o gerenciamento do código-fonte e o trabalho em equipe, todas as ferramentas, scripts de migração e arquivos gerados foram versionados e estão disponíveis publicamente no GitHub através do link: <https://github.com/thalesfrts/Projeto-NoSQL-LibAcad>

A proposta busca demonstrar as diferenças entre os paradigmas relacional e NoSQL, analisando como tabelas, relacionamentos e entidades podem ser transformados em coleções e documentos JSON.

Além disso, o trabalho apresenta:

- Modelagem NoSQL;
- Estratégias de desnormalização;
- Scripts de migração;

- Implementação no MongoDB;
- Operações CRUD;
- Consultas utilizando Aggregation Framework;
- Comparação crítica entre os modelos.

1 - Descrição do banco relacional original.

Tabela 1 - Estrutura do banco relacional

Entidade	Função	Atributos e Tipo de Dados
Usuario	Armazena usuário	id_usuario (INT), nome (VARCHAR), email (VARCHAR), tipo_usuario (VARCHAR)
Categoria	Classifica os livros por assunto	id_categoria (INT), nome (VARCHAR)
Autor	Armazena autores	id_autor (INT), nome (VARCHAR)
Livro	Armazena os livros	id_livro (INT), titulo (VARCHAR), ano (INT), quantidade_estoque (INT), id_categoria (INT)
Livro_Autor	Relaciona livros e autores	id_livro (INT), id_autor (INT)
Emprestimo	Registra empréstimo	id_emprestimo (INT), id_usuario (INT), data_emprestimo (DATE), data_prevista (DATE), status (VARCHAR)
ItemEmprestimo	Registra livro emprestado	id_item (INT), id_emprestimo (INT), id_livro (INT), data_devolucao (DATE), multa (DECIMAL)

O banco relacional original possui os seguintes relacionamentos:

- Usuário realiza vários empréstimos;
- Um empréstimo possui vários itens;
- Um livro pode estar em vários itens de empréstimo;
- Uma categoria possui vários livros;
- Um livro pode possuir vários autores;
- Um autor pode escrever vários livros.

O banco relacional apresenta forte normalização, reduzindo redundâncias e garantindo integridade referencial.

Entretanto, algumas limitações podem ser observadas:

- Necessidade de JOINS frequentes;
- Maior complexidade em consultas;
- Estrutura rígida;
- Dificuldade de escalabilidade horizontal.

Exemplo:

Para listar um empréstimo completo, seria necessário realizar JOIN entre:

- Empréstimo;
- ItemEmprestimo;
- Livro;
- Usuário.

No MongoDB, parte dessas informações pode ser agrupada em um único documento.

1.1 - Casos de Uso e Operações Frequentes no Sistema Original

Para compreender a dinâmica de funcionamento da biblioteca acadêmica e mapear como as entidades interagem na prática, foram identificados os principais casos de uso operados rotineiramente no sistema relacional: * Autenticação e Controle de

Acesso: O sistema identifica o perfil do usuário conectado através do campo `tipo_usuario` , diferenciando as permissões e telas visíveis para "Alunos" e "Administradores".

Consulta e Busca de Acervo: Operação altamente frequente em que usuários buscam livros por meio de filtros como título, filtragem por assunto através da entidade Categoria ou por meio de vínculos de autoria na tabela `Livro_Autor`.

Fluxo de Realização de Empréstimos: Um usuário ("Aluno") solicita a retirada de exemplares. O sistema verifica a `quantidade_estoque` do livro, cria um registro na tabela `Emprestimo` contendo a `data_emprestimo` e calcula automaticamente a `data_prevista` para devolução, marcando o status inicial do processo como "Ativo".

Gerenciamento de Itens Empréstados: Mapeia os múltiplos livros que compõem uma única retirada através da tabela `ItemEmprestimo`. Cada item recebe um controle individual para monitorar quais volumes já retornaram.

Devolução de Exemplares e Aplicação de Penalidades: No ato da devolução de um livro, a `data_devolucao` é preenchida no registro do item. O sistema compara essa data com a `data_prevista` ; caso haja atraso, calcula-se o valor acumulado no campo `multa` e o status do empréstimo é atualizado.

Manutenção do Estoque (Operação Administrativa): Usuários do tipo "Administrador" realizam a inserção de novos títulos ou atualizam manualmente a volumetria de unidades disponíveis em `quantidade_estoque`.

2 - Diagrama ER

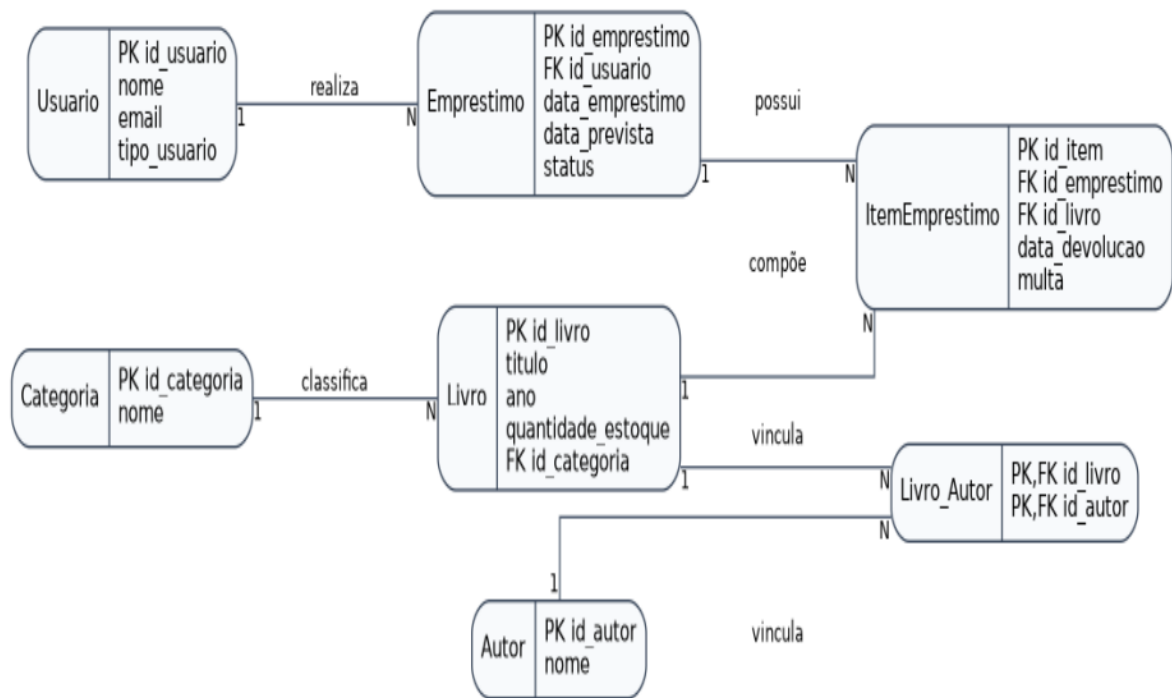


Figura 1 - Diagrama ER

3 - Modelo NoSQL proposto

Na conversão do banco relacional para o MongoDB, os tipos de dados nativos do SQL Server foram mapeados para o padrão BSON.

1. **Numéricos (INT/DECIMAL):** Os campos numéricos originais, como quantidades e IDs, passam a ser armazenados como Int32 ou Double.
2. **Textos (VARCHAR):** Campos descritivos (como nomes de usuários, títulos de livros e status) são convertidos diretamente para o tipo String.
3. **Datas (DATE):** Campos de data de empréstimo, previsão e devolução utilizam o tipo Date nativo do MongoDB (ISODate).
4. **Relacionamentos e Chaves Estrangeiras:** As chaves estrangeiras (FK) que antes referendavam IDs de outras tabelas deixam de existir isoladamente. Em vez de cruzar tabelas, aplicamos o padrão de *Embedding* (documentos embutidos). Dados como a categoria do livro e os itens do empréstimo passam a ser objetos e listas (arrays) armazenados hierarquicamente dentro do próprio documento principal, eliminando a necessidade de JOINS.

Na modelagem NoSQL, o objetivo principal foi reduzir JOINS e aproximar dados frequentemente utilizados juntos. Foram utilizadas duas abordagens principais: **Embedding** (documentos embutidos), que teve como justificativa o fato de os dados serem pequenos e acessados com muita frequência. Sendo assim será utilizado em:

- itens de empréstimo;
- autores do livro;
- categoria do livro.

Referencing (referência entre documentos), que teve como justificativa evitar a duplicação demasiada de grandes volumes de dados. É utilizado para:

- usuário e empréstimo;
- livro e empréstimo.

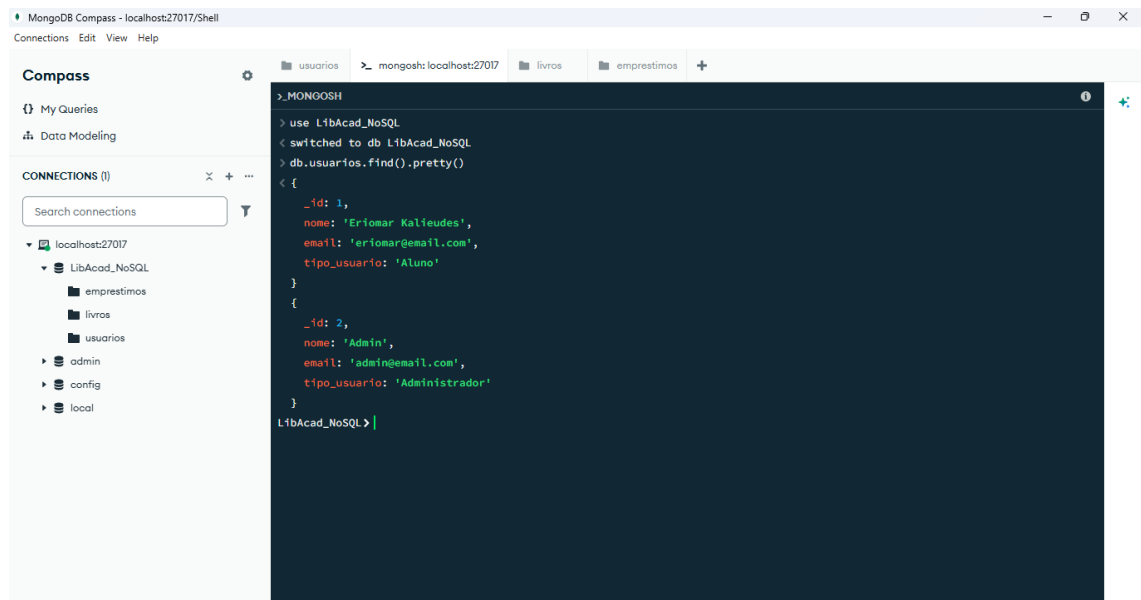
Além do mais foi utilizado o padrão One-to-Many Inside the Parent para os empréstimos, pois os itens sempre serão acessados junto aos empréstimos, ou seja, além de consultar os empréstimos, teremos as informações dos livros, usuários, categoria etc. Dessa forma não será necessário utilizar Joins, a consulta será simplificada e o desempenho melhorado.

Diferente do banco relacional, foi necessário definir algumas coleções para o banco NoSQL. As coleções escolhidas foram:

- usuários – que irá armazenar os usuários no banco
- livros – que irá armazenar os livros, autores e categorias
- empréstimos – que irá armazenar os empréstimos e seus itens

4 - Exemplos de documentos finais (JSON)

Abaixo segue os exemplos de como ficaram os documentos em Json para cada uma das coleções escolhidas.



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows a connection to 'localhost:27017' with a database named 'LibAcad_NoSQL'. The 'usuarios' collection is selected. The main panel displays a JSON document for a user. The document is as follows:

```
> MONGODB
> use LibAcad_NoSQL
< switched to db LibAcad_NoSQL
> db.usuarios.find().pretty()
< {
  "_id": 1,
  "nome": "Eriomar Kalieudes",
  "email": "eriomargemail.com",
  "tipo_usuario": "Aluno"
}
{
  "_id": 2,
  "nome": "Admin",
  "email": "admin@email.com",
  "tipo_usuario": "Administrador"
}
LibAcad_NoSQL>
```

Figura 2 – Documento Json para Usuários.

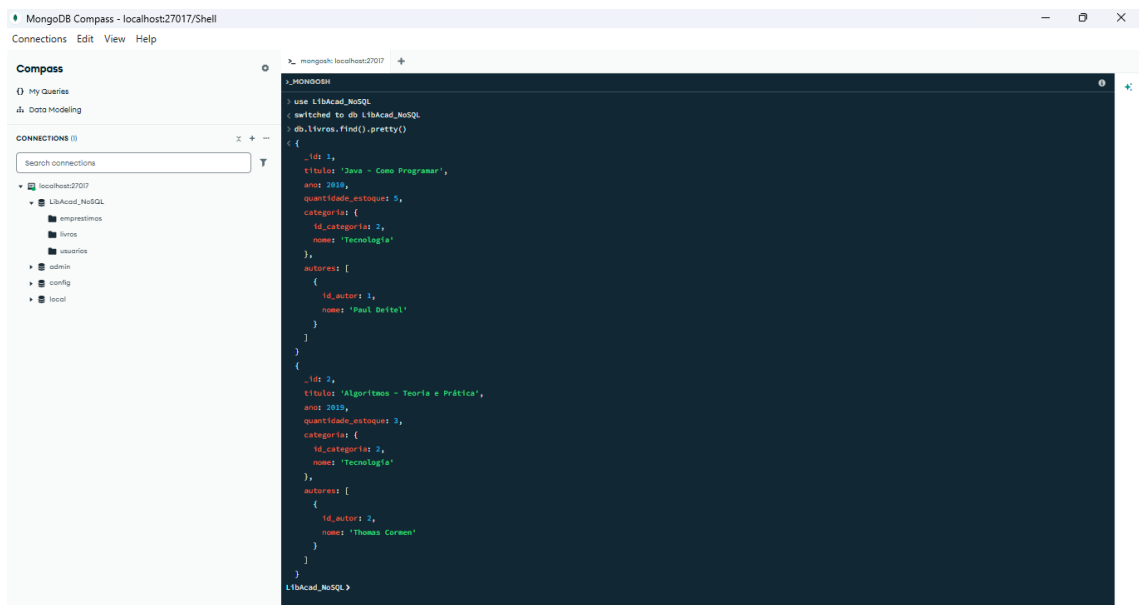


Figura 3 – Documento Json para Livro

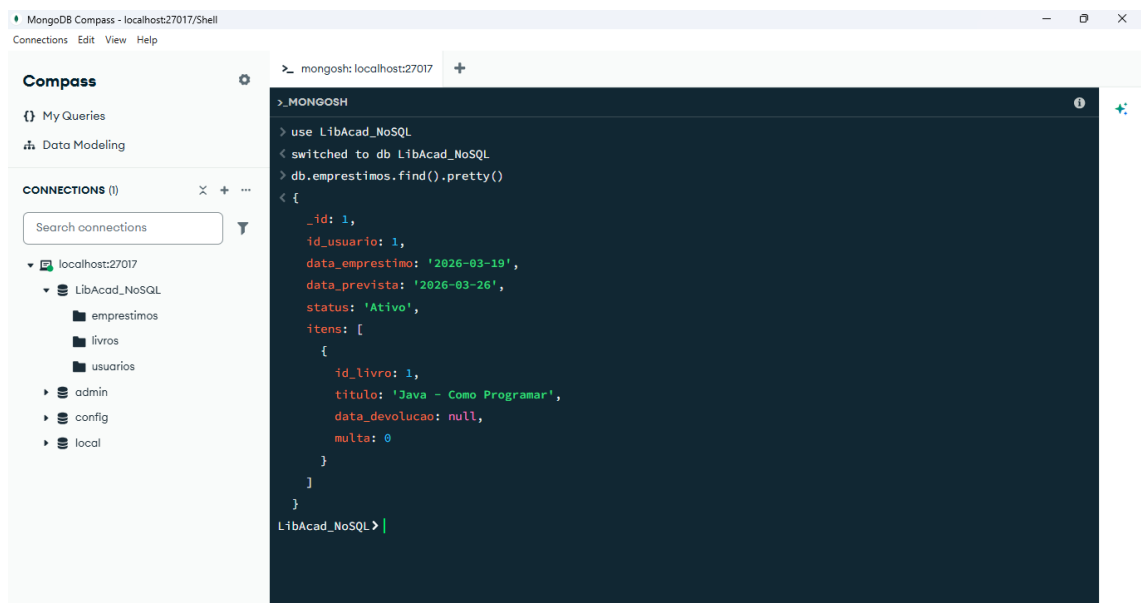


Figura 4 – Documento Json para Empréstimos

Para assegurar a integridade referencial durante o processo de migração, estabeleceu-se a conversão direta da Chave Primária (Primary Key) do modelo relacional para o campo `_id` nativo do MongoDB. Esta estratégia de mapeamento de identidade garante que a unicidade dos registros seja preservada, permitindo que as relações de Referencing entre as coleções sejam mantidas de forma consistente, eliminando ambiguidades e otimizando a indexação automática realizada pelo motor do MongoDB.

Por fim, foram definidos alguns índices com o objetivo de melhorar o desempenho das consultas. São eles:

- Índice por email:

```
db.usuarios.createIndex({ email: 1 })
```

- Índice por título

```
db.livros.createIndex({ titulo: 1 })
```

- Índice por status do empréstimo

```
db.emprestimos.createIndex({ status: 1 })
```

- Índice por usuário

```
db.emprestimos.createIndex({ id_usuario: 1 })
```

5 - Scripts de conversão e carregamento

A extração dos dados foi realizada executando alguns scripts no banco de dados relacional do SQL Server. As informações foram extraídas das principais tabelas do banco usando *select*. Segue abaixo os comandos utilizados.

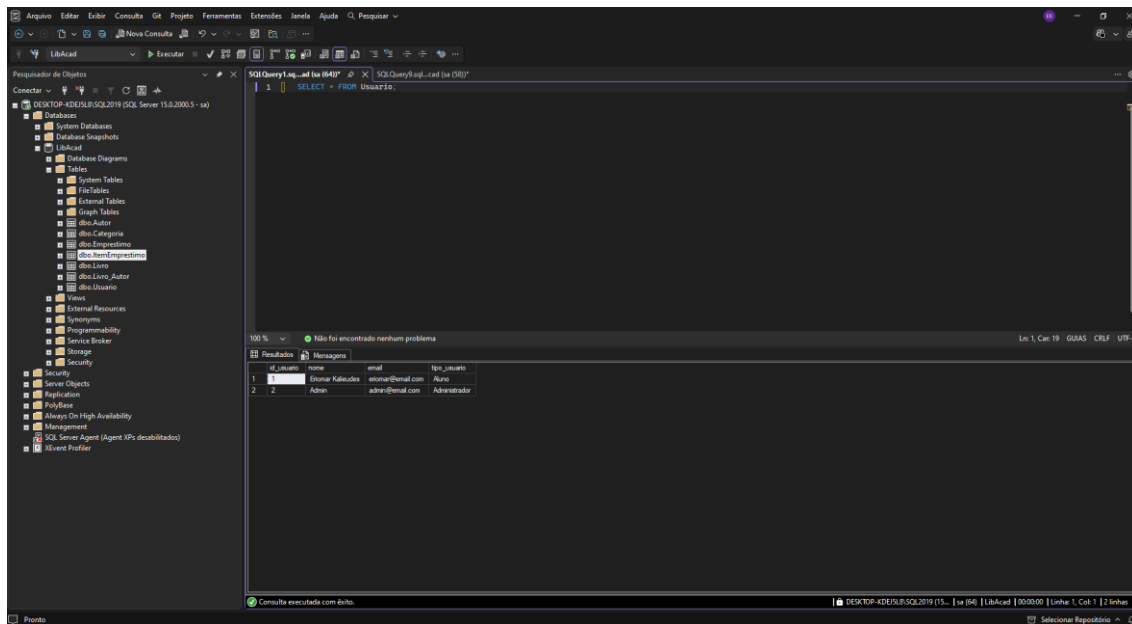


Figura 5 – Select de usuários.

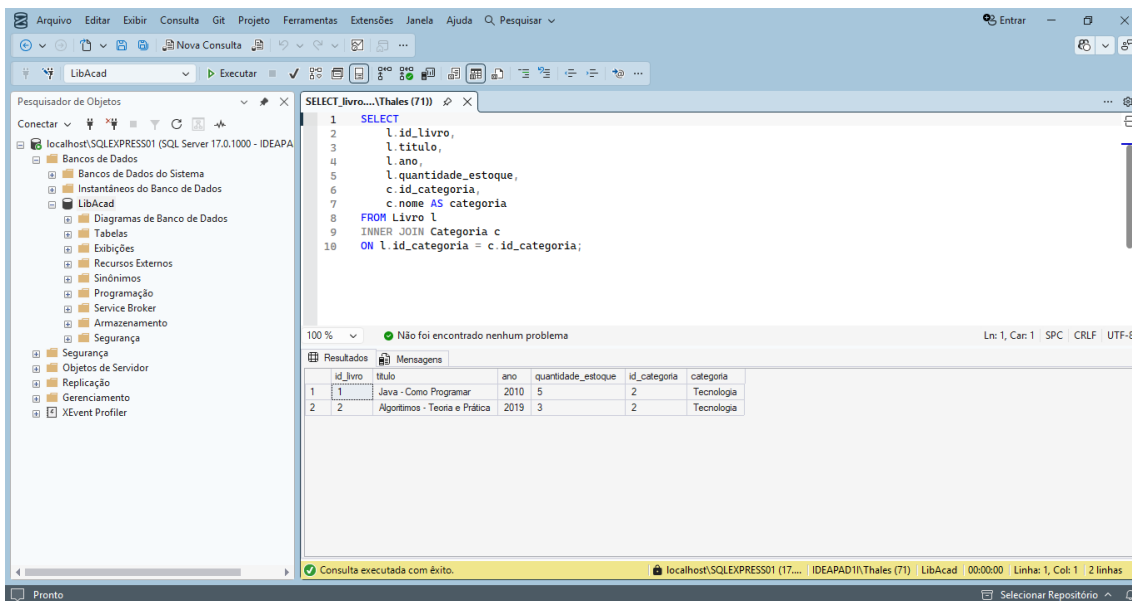


Figura 6 – Select de livros.

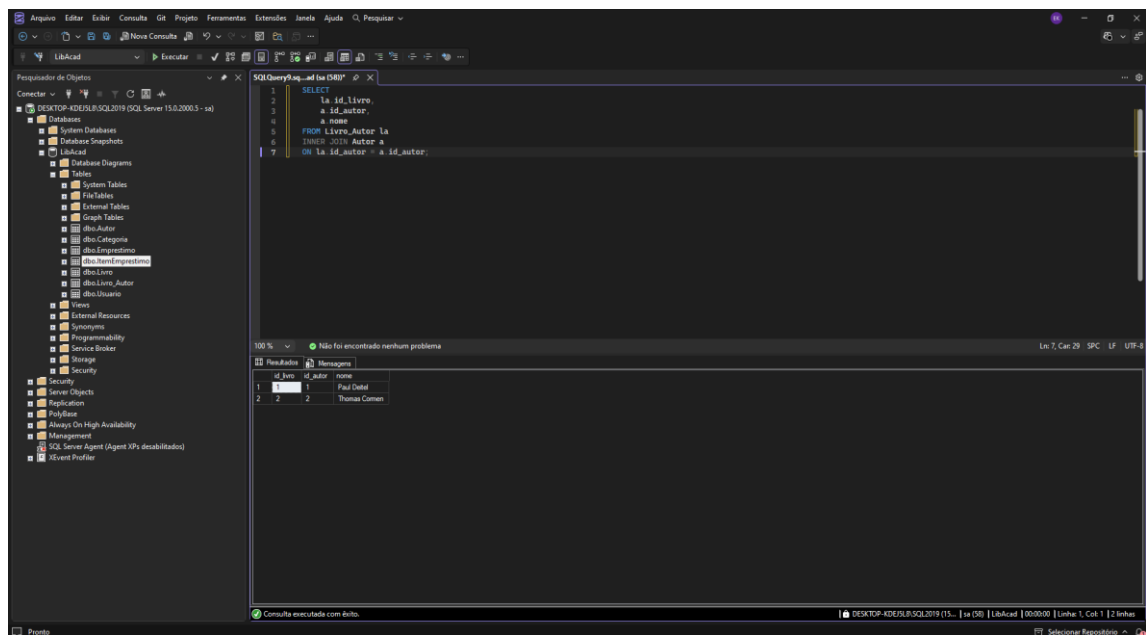


Figura 7 – Select de autores livros.

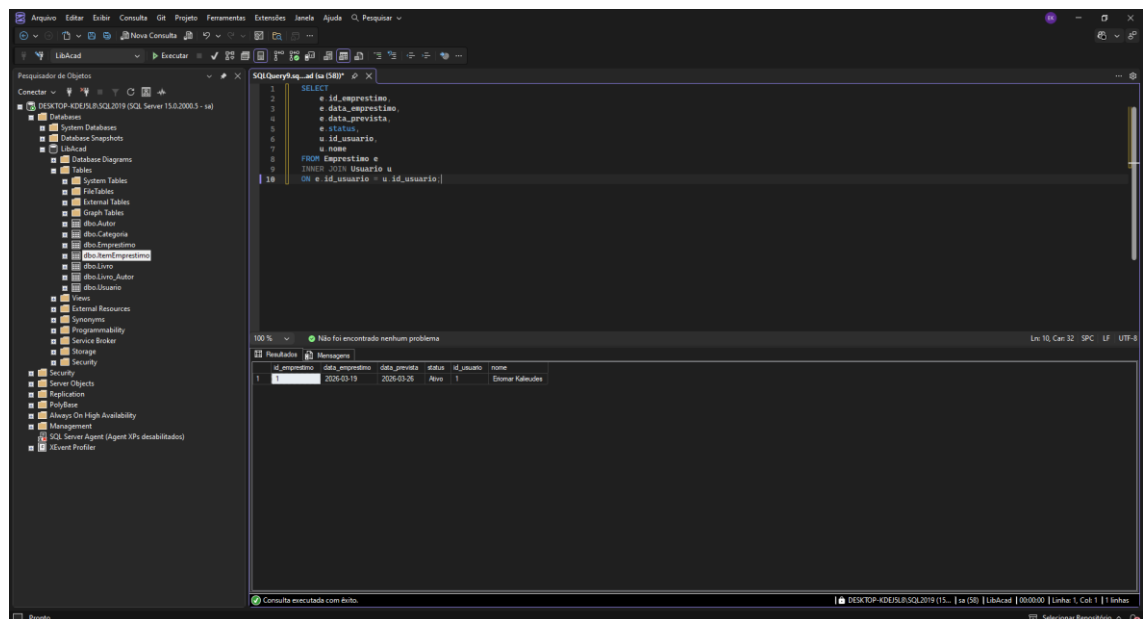


Figura 8 – Select de empréstimos.

Após a extração dos dados do banco relacional, eles foram processados e convertidos para o formato .json aplicando o padrão de documentos embutidos (Embedding) definido na modelagem. Para cada uma das coleções resultantes que serão importadas no MongoDB, obteve-se a seguinte estrutura:

Usuários: [

```
{  
  
  "_id": 1,  
  
  "nome": "Eriomar Kalieudes",  
  
  "email": "eriomar@email.com",  
  
  "tipo_usuario": "Aluno"  
},
```

```
{  
  
  "id_usuario": 2,  
  
  "nome": "Admin",  
  
  "email": "admin@email.com",  
  
  "tipo_usuario": "Administrador"  
}
```

]

Livro: [

```
{  
  
  "_id": 1,  
  
  "titulo": "Java - Como Programar",  
  
  "ano": 2010,
```

```
"quantidade_estoque": 5,

"categoria": {

  "id_categoria": 2,

  "nome": "Tecnologia"

},

"autores": [

  {

    "id_autor": 1,

    "nome": "Paul Deitel"

  }

]

},

{

  "id_livro": 2,

  "titulo": "Algoritmos - Teoria e Prática",

  "ano": 2019,

  "quantidade_estoque": 3,

  "categoria": {

    "id_categoria": 2,
```

```
    "nome": "Tecnologia"

  },

  "autores": [

    {

      "id_autor": 2,

      "nome": "Thomas Cormen"

    }

  ]

}
```

```
Empréstimo: Empréstimo : [

  {

    "_id": 1,

    "id_usuario": 1,

    "data_emprestimo": "2026-03-19",

    "data_prevista": "2026-03-26",

    "status": "Ativo",

    "itens": [

      {
```

```

    "id_item": 1,

    "id_livro": 1,

    "titulo": "Java - Como Programar",

    "data_devolucao": null,

    "multa": 0.0

  }

]

}

]

```

Para mostrar que os dados do banco NoSQL estão de acordo com os do banco relacional, foram executados alguns comandos em cada um dos bancos mostrando e comparando que todas as quantidades são iguais. Segue as imagens abaixo.

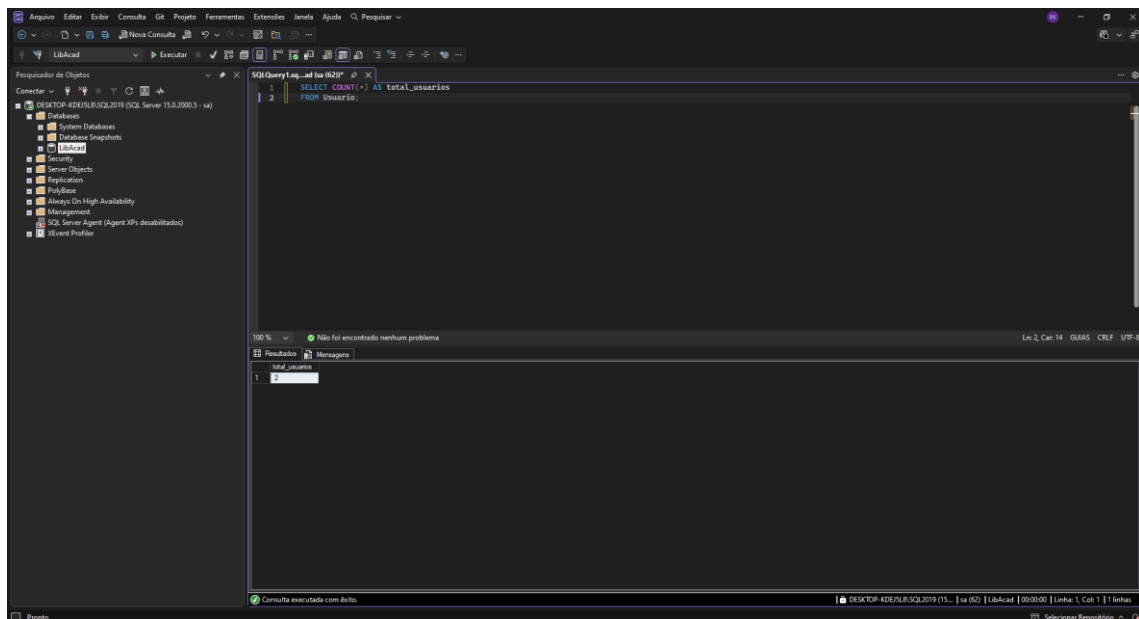


Figura 9 - Usuários SQL.

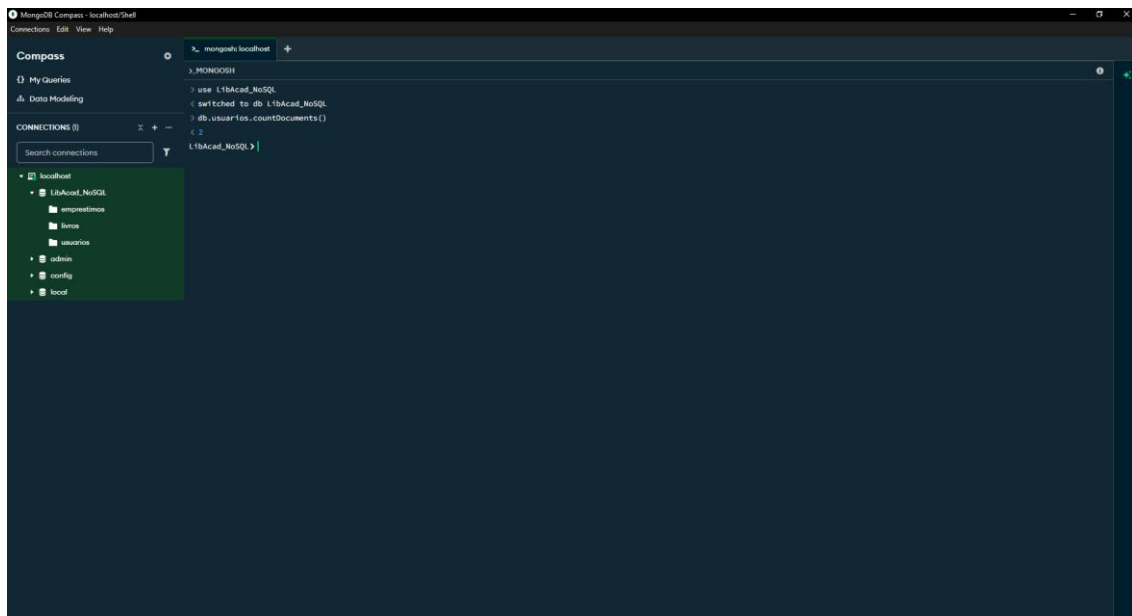


Figura 10 - Usuários NoSQL.

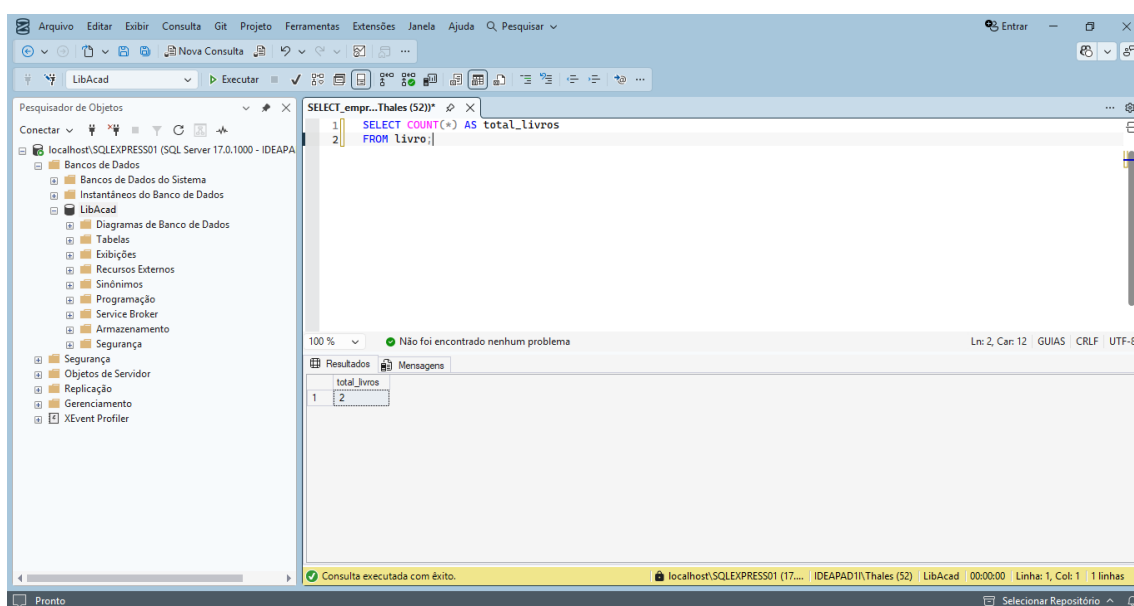


Figura 11 - Livros SQL.

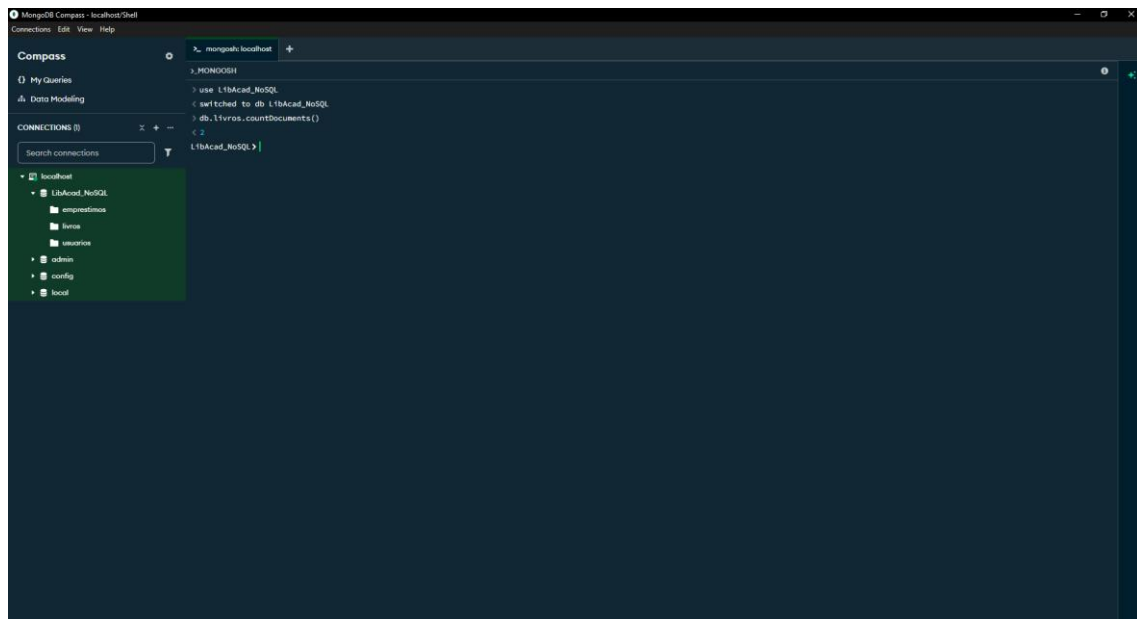


Figura 12 - Livros NoSQL.

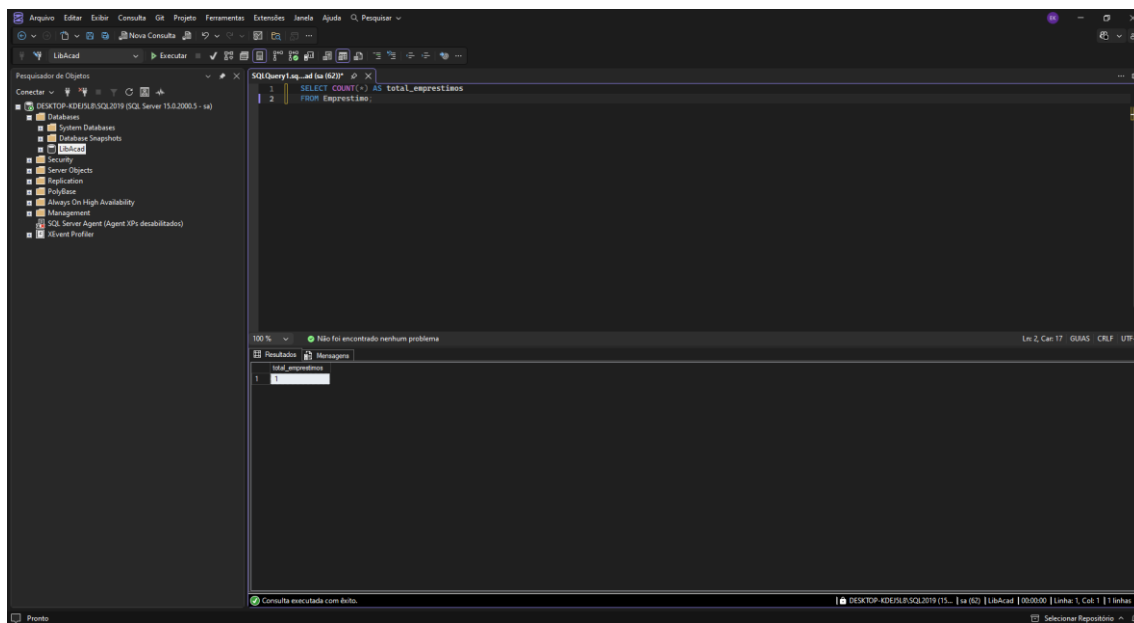


Figura 13 - Empréstimo SQL.

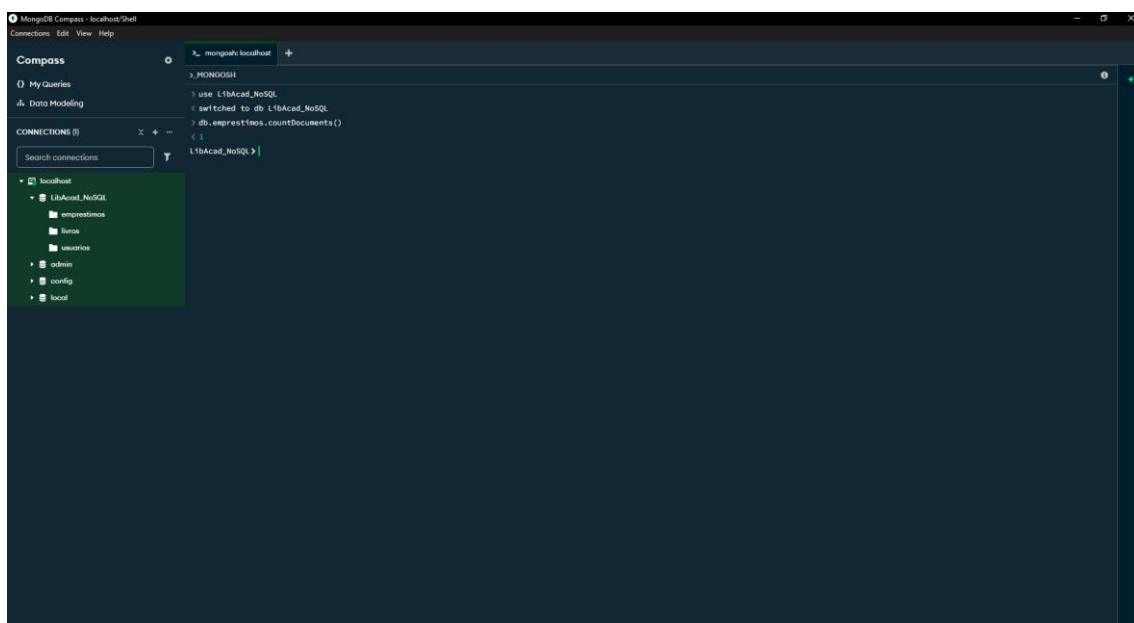


Figura 14 - Empréstimo NoSQL.

A criação do banco foi feita de forma manual apenas criando uma conexão informando o nome e posteriormente, dentro da conexão, o banco. Esse procedimento é simples e intuitivo, basta clicar no sinal de adicionar e preencher as informações necessárias.

Diferentemente do banco, as coleções foram criadas utilizando alguns comandos pelo terminal do mongoDB (MONGOSH). Para a coleção de usuário foi utilizado o seguinte comando:

```
db.createCollection("usuarios")
```

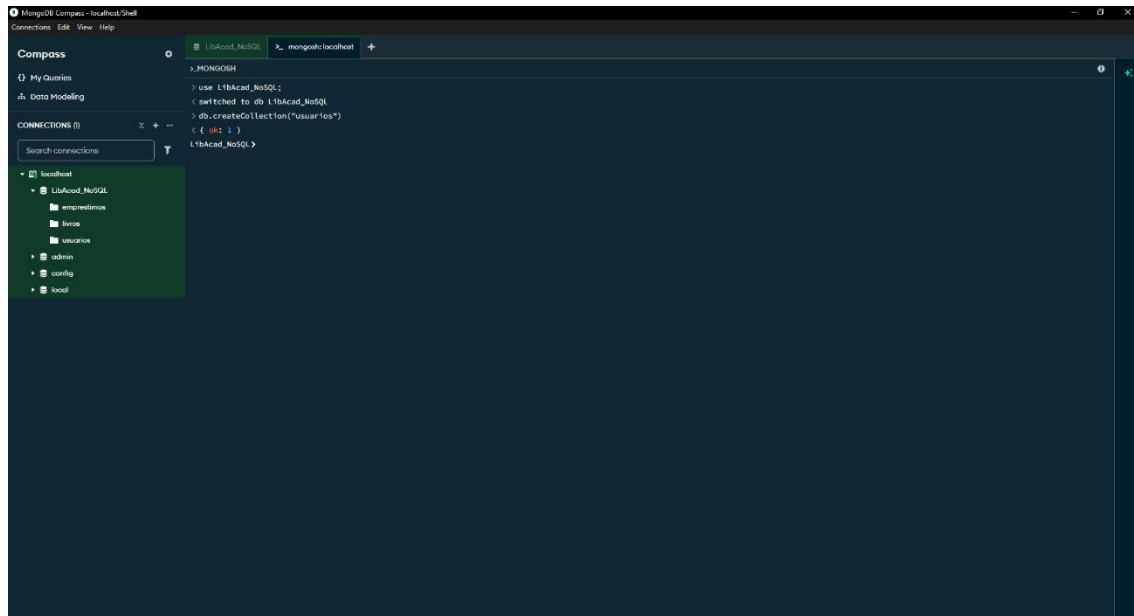


Figura 15 - Coleção de usuários.

Na criação da coleção livros foi utilizado o seguinte comando:

```
db.createCollection("livros")
```

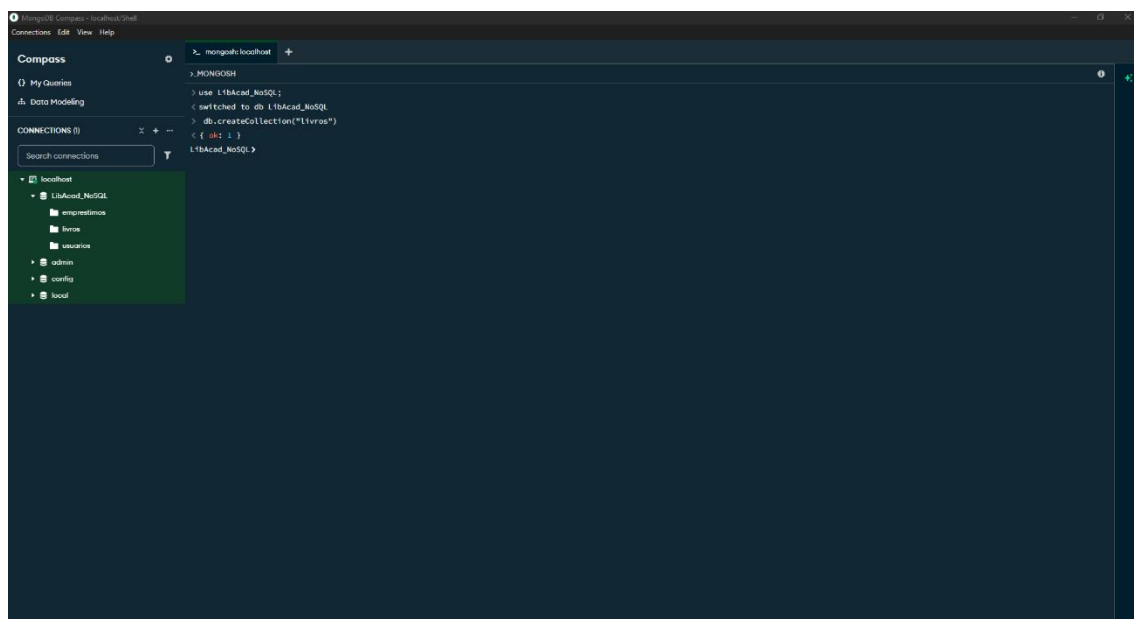


Figura 16 - Coleção de livros.

E por fim, a coleção de empréstimos foi criada da seguinte forma:

```
db.createCollection("emprestimos")
```

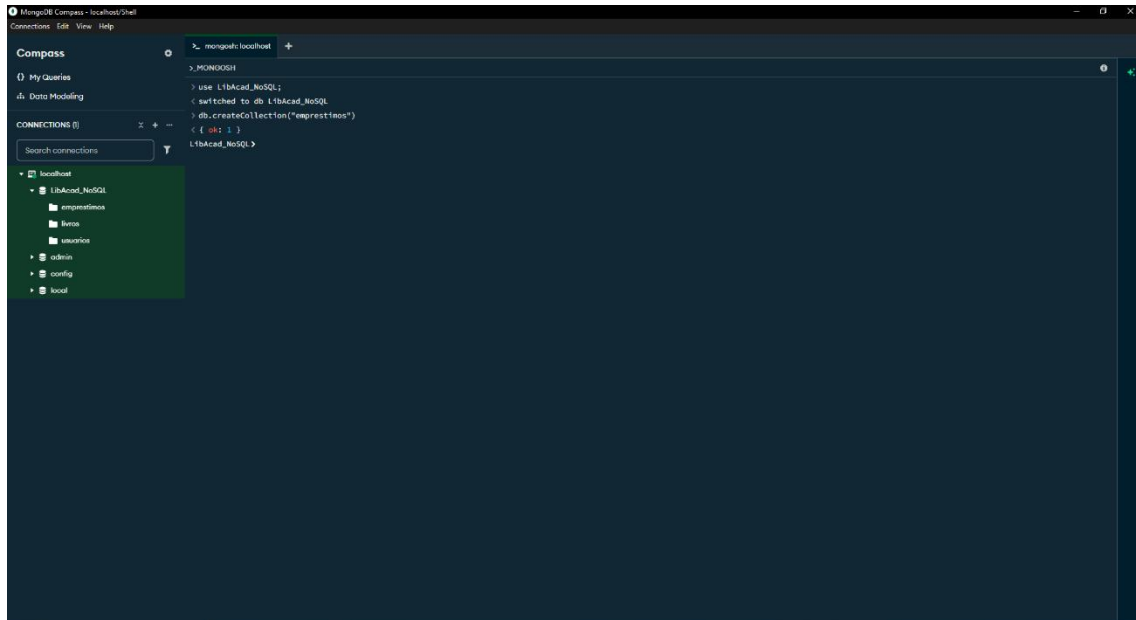


Figura 17 - Coleção de empréstimos.

Para realizar a migração e o preenchimento do banco NoSQL de forma automatizada e fiel ao novo esquema, os documentos estruturados em JSON foram inseridos nas respectivas coleções através de scripts de carregamento nativos do MongoDB, utilizando o comando `insertMany`. Essa abordagem garante que os dados convertidos do modelo relacional sejam integrados em lote, mantendo a integridade e a consistência das regras de *Embedding* (dados embutidos) e *Referencing* (referências) projetadas durante a modelagem.

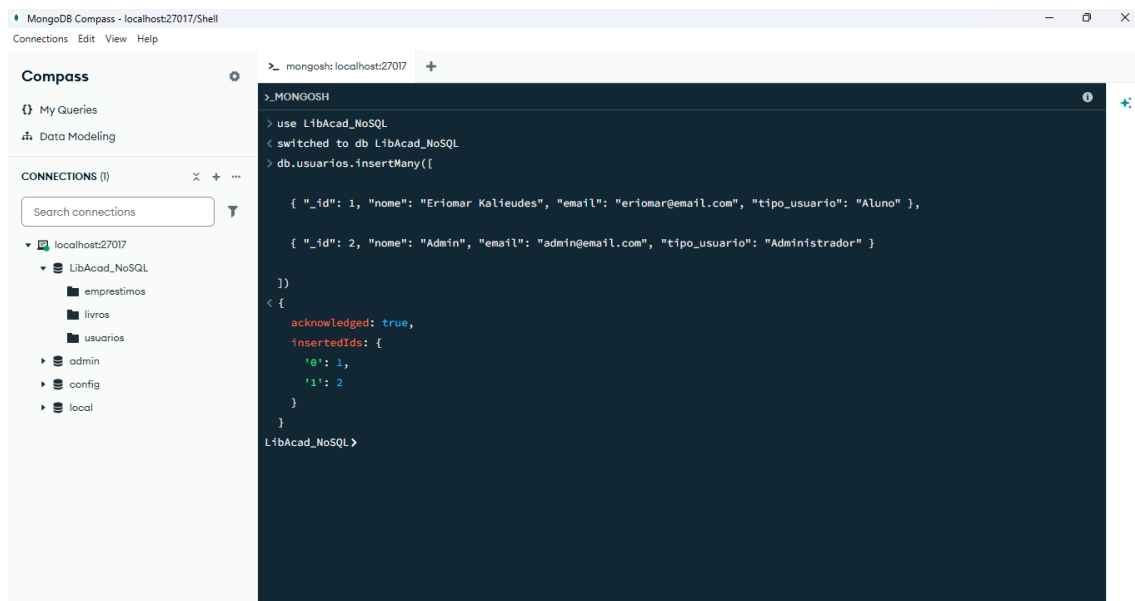


Figura 188 - Script de carregamento da coleção de usuários.

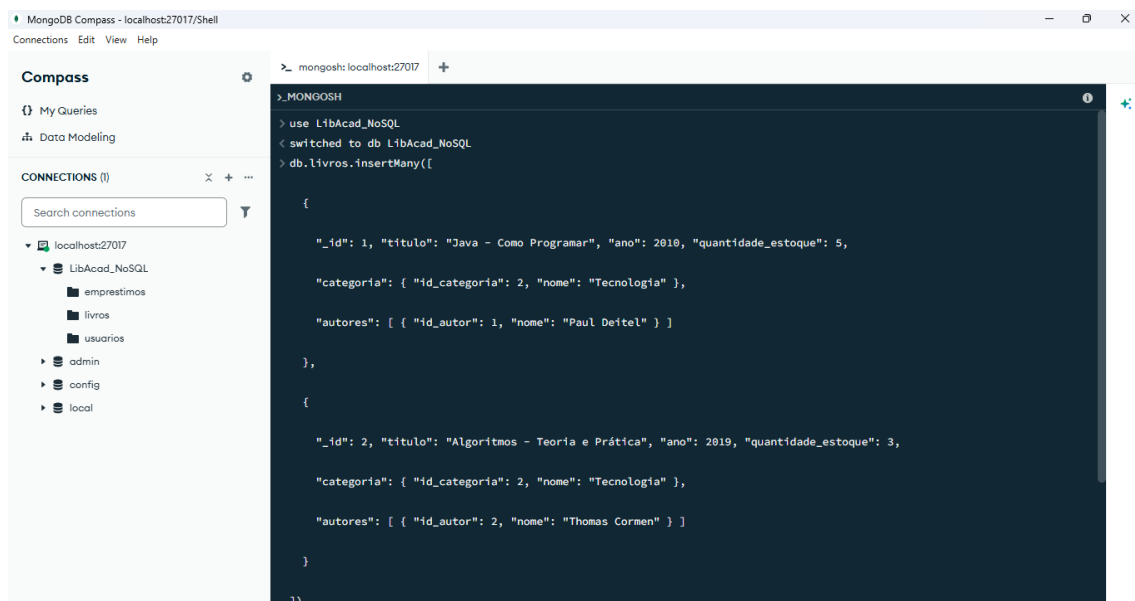


Figura 19 - Script de carregamento da coleção de livros (com categorias e autores embutidos) – Parte 1.

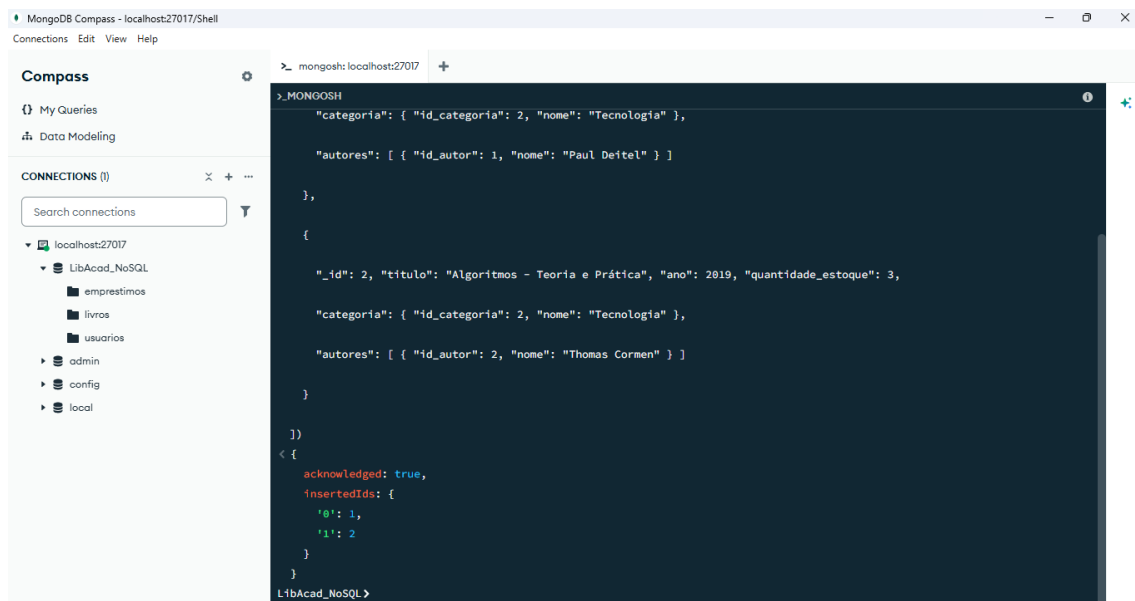


Figura 20 - Script de carregamento da coleção de livros (com categorias e autores embutidos) – Parte 2.

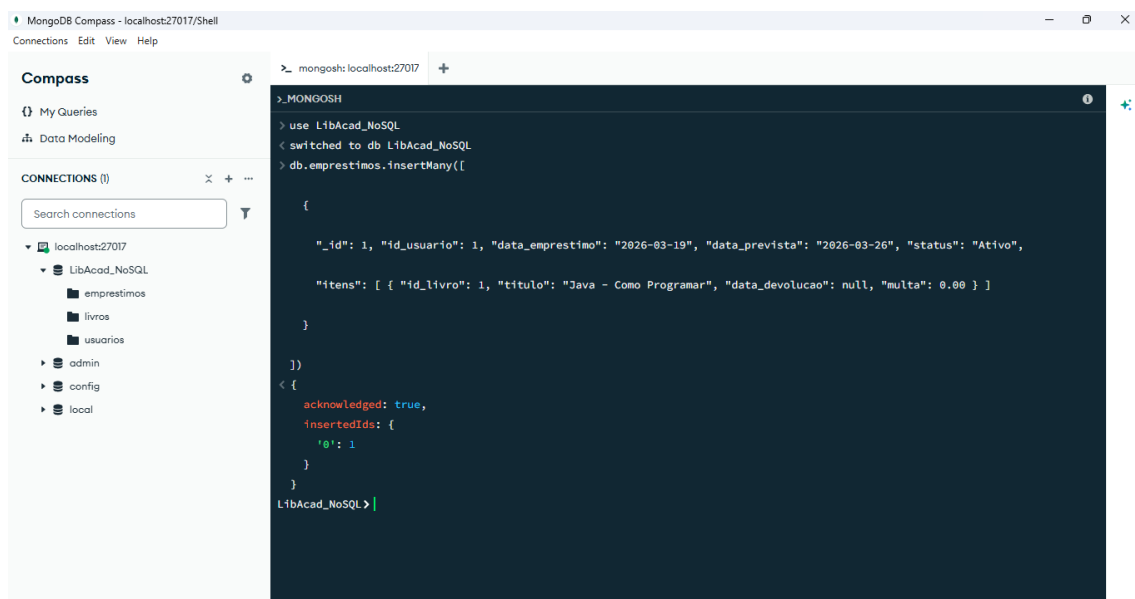


Figura 21 - Script de carregamento da coleção de empréstimos.

6 - Consultas realizadas e resultados

Também foram implementadas as operações básicas CRUD (create, read, update e delete). Segue abaixo a imagem de cada uma das operações e seus resultados.

Create

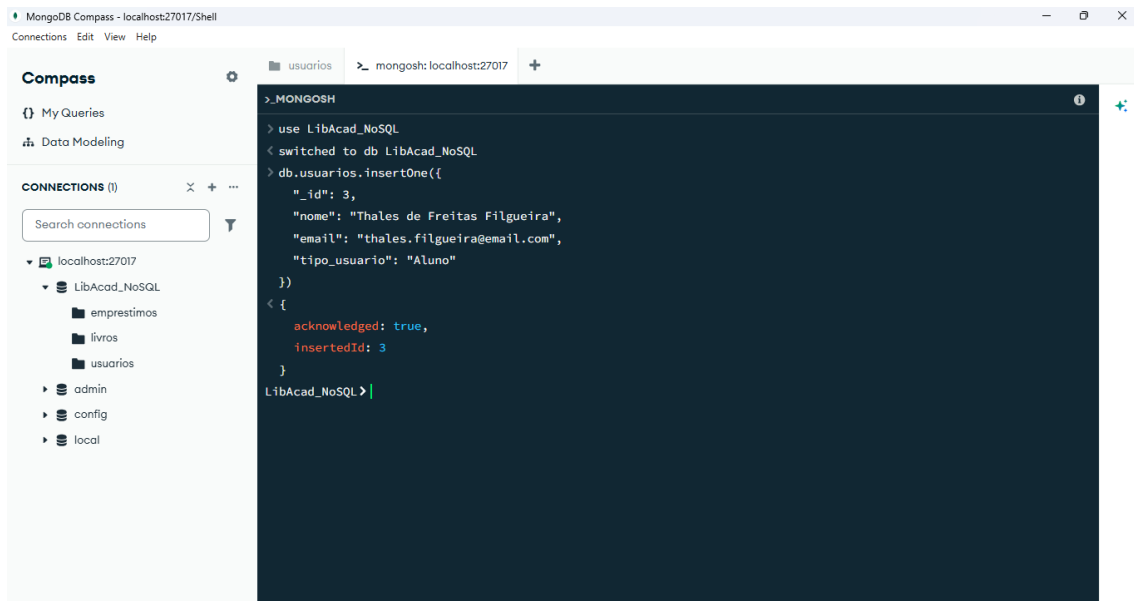


Figura 22 - Criando usuário.

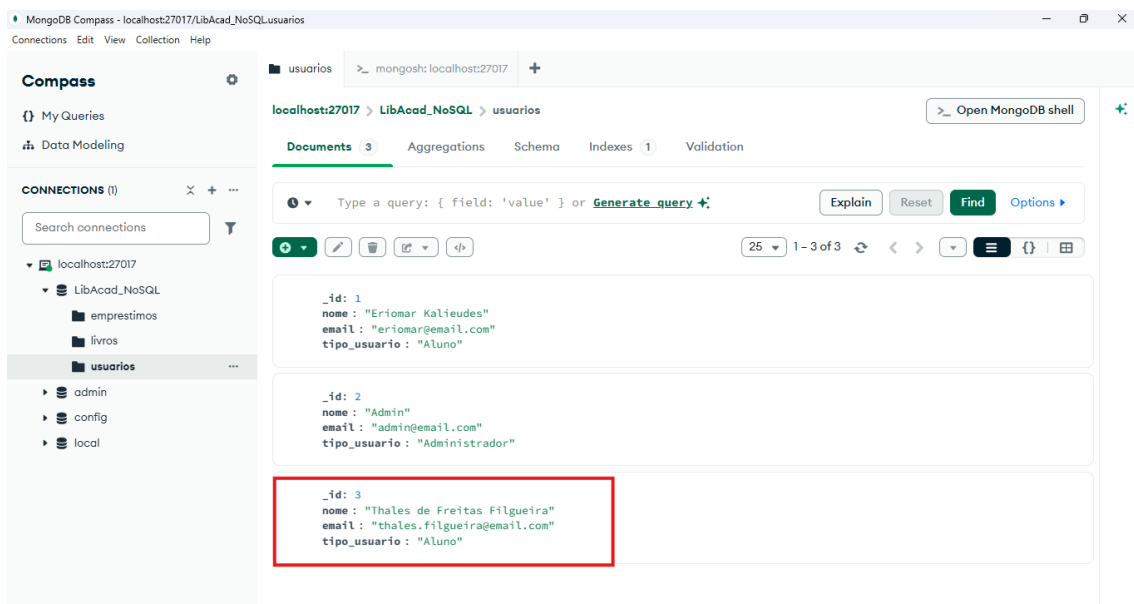


Figura 23 - Resultado do create.

Read

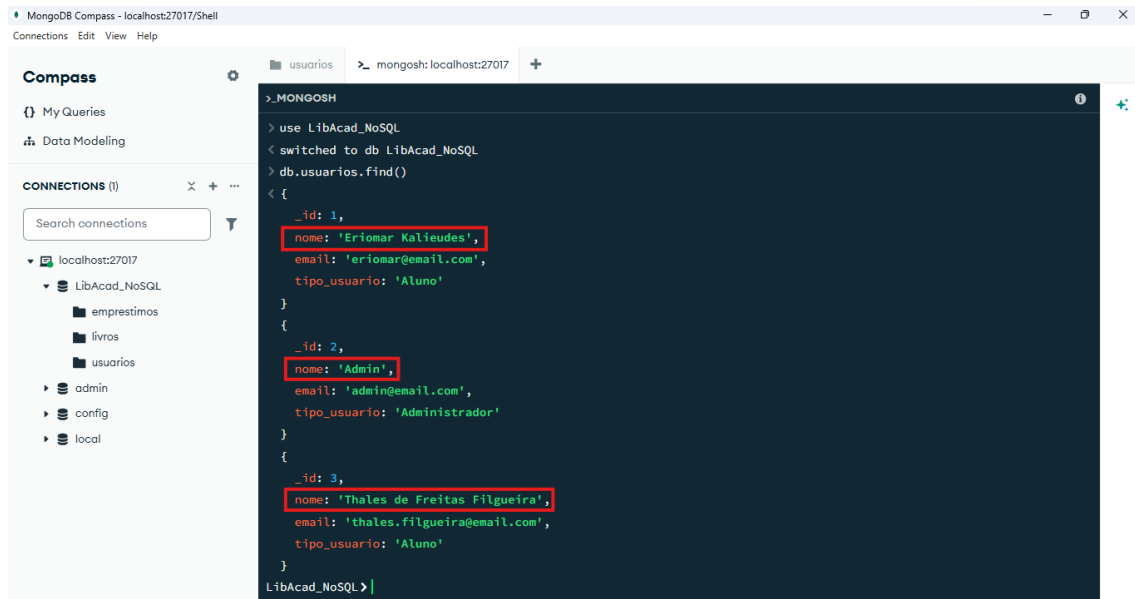


Figura 24 - Resultado do read.

Update

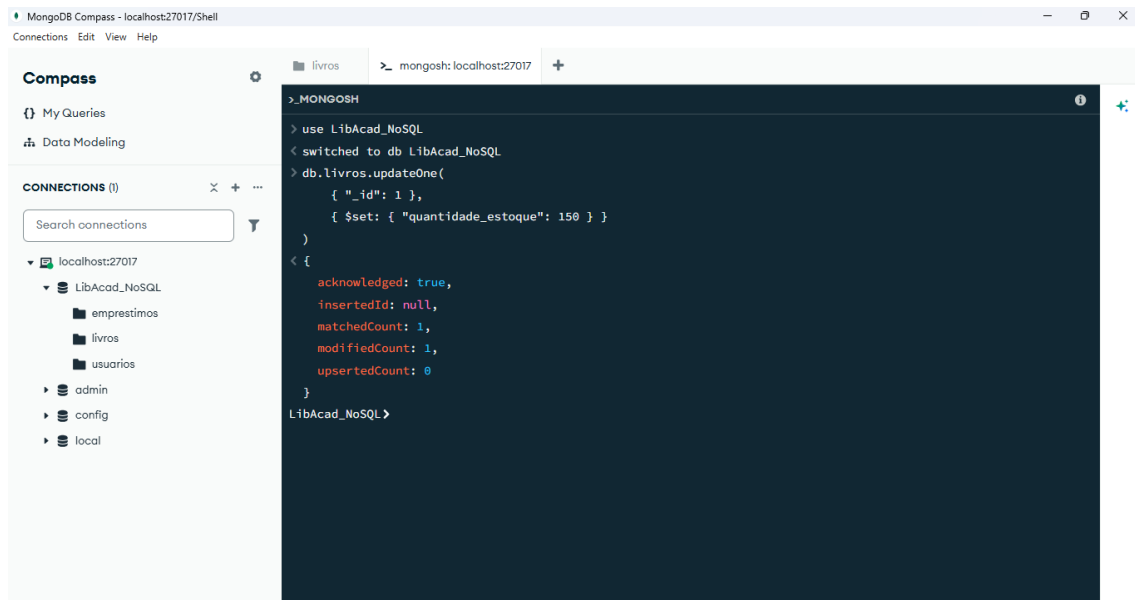


Figura 25 - Update de livro.

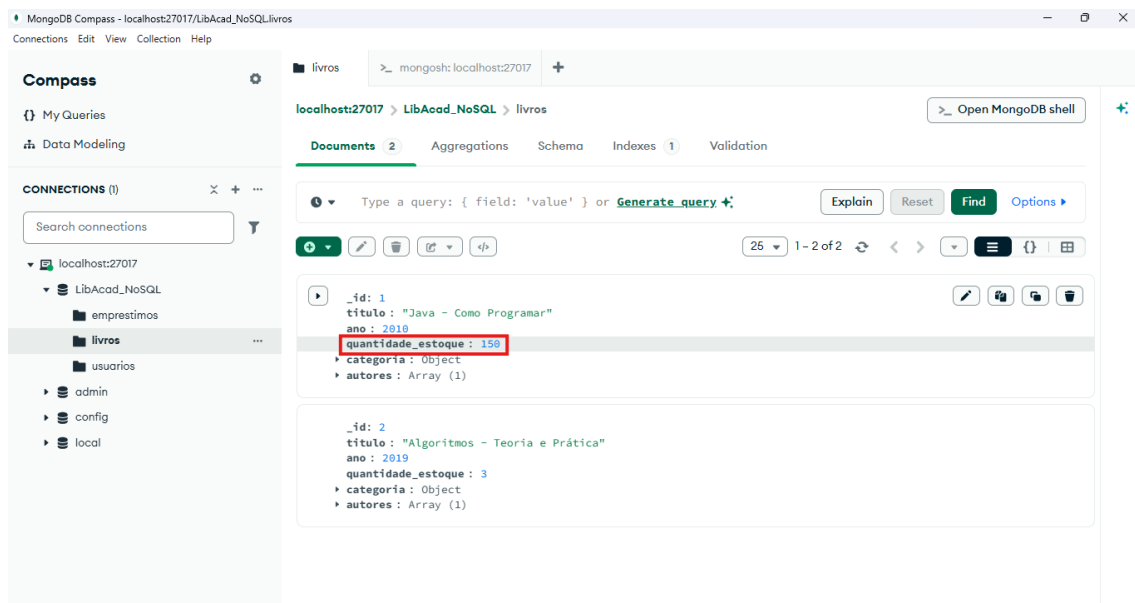


Figura 26 – Resultado do update.

Delete

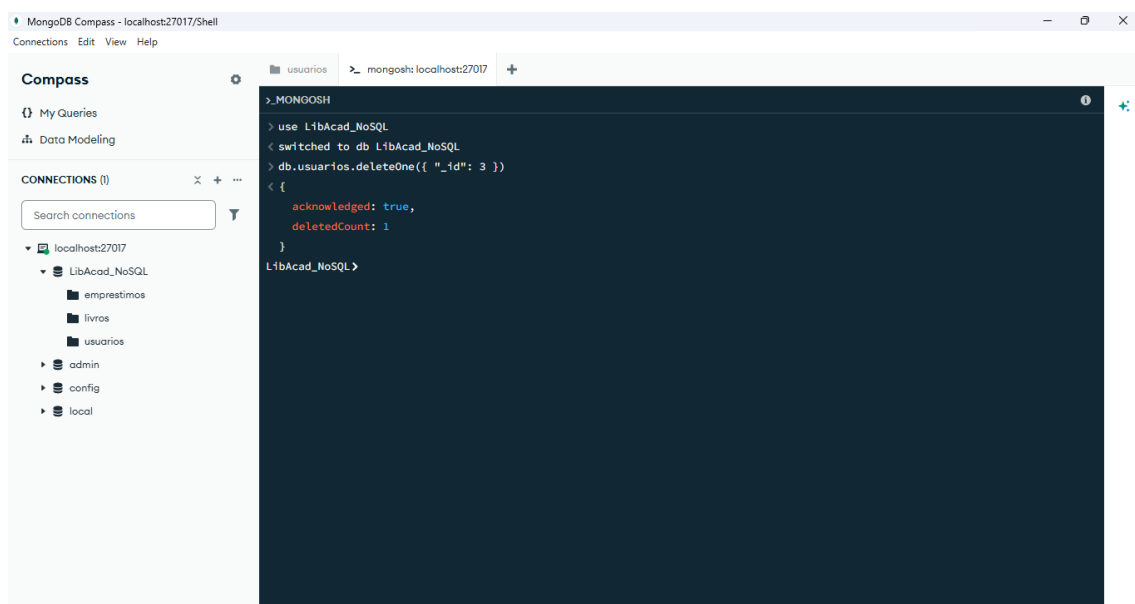


Figura 27 - Delete de usuário.

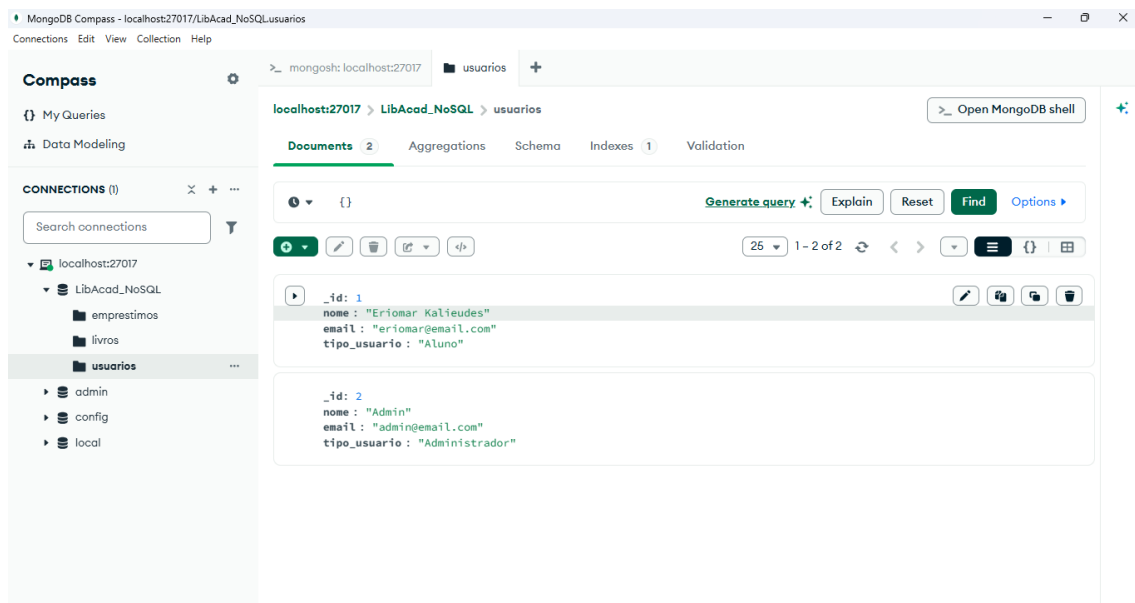


Figura 28 – Resultado do delete.

As consultas simples com filtro, projeções e ordenações ficaram da seguinte forma juntamente com a agregação.

Filtros

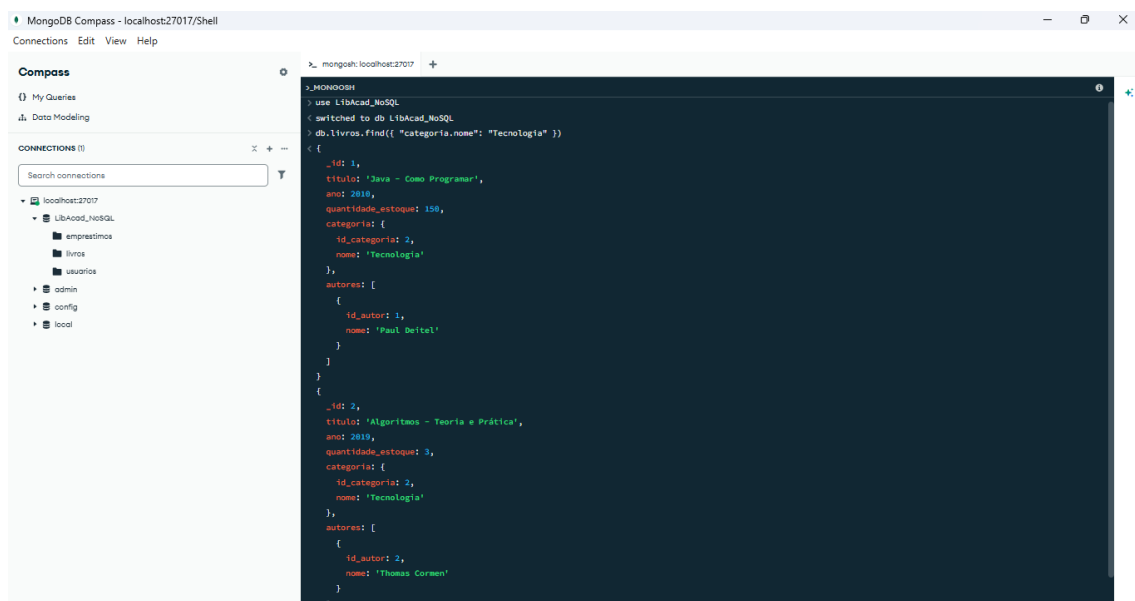


Figura 29 - Filtrando por categoria.

Projeção

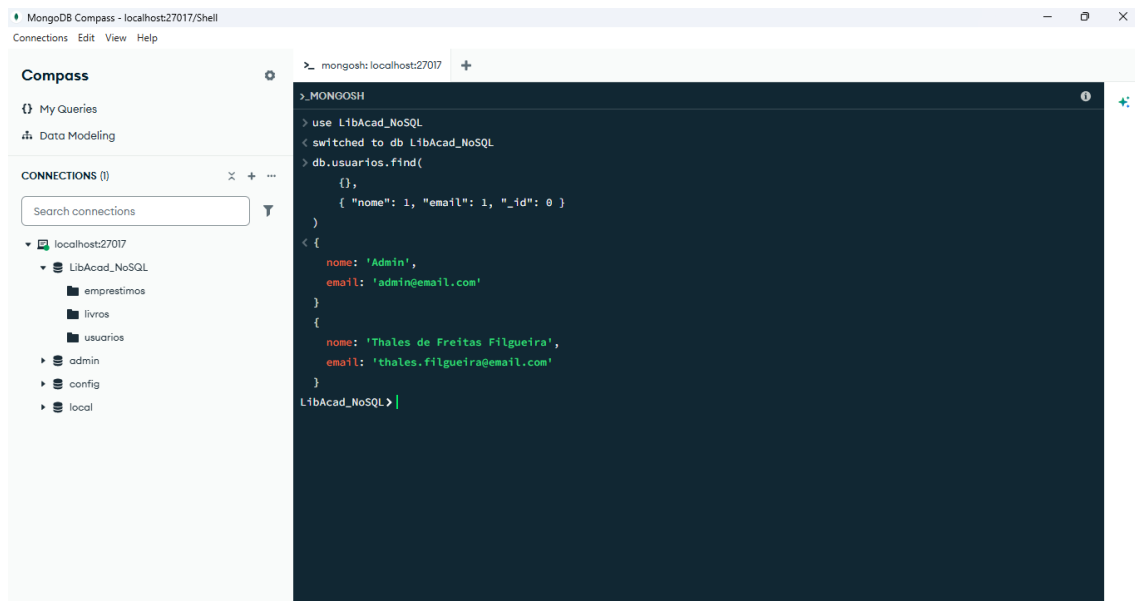


Figura 30 - Projeção de dados selecionados.

Ordenação

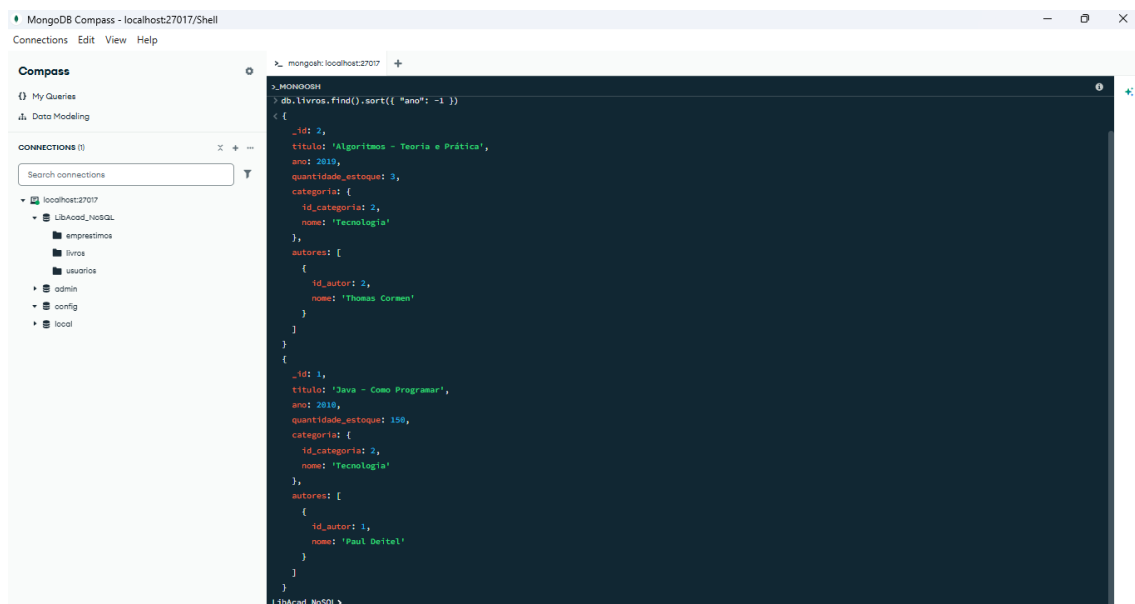


Figura 31 - Ordenação cronológica.

Agregações

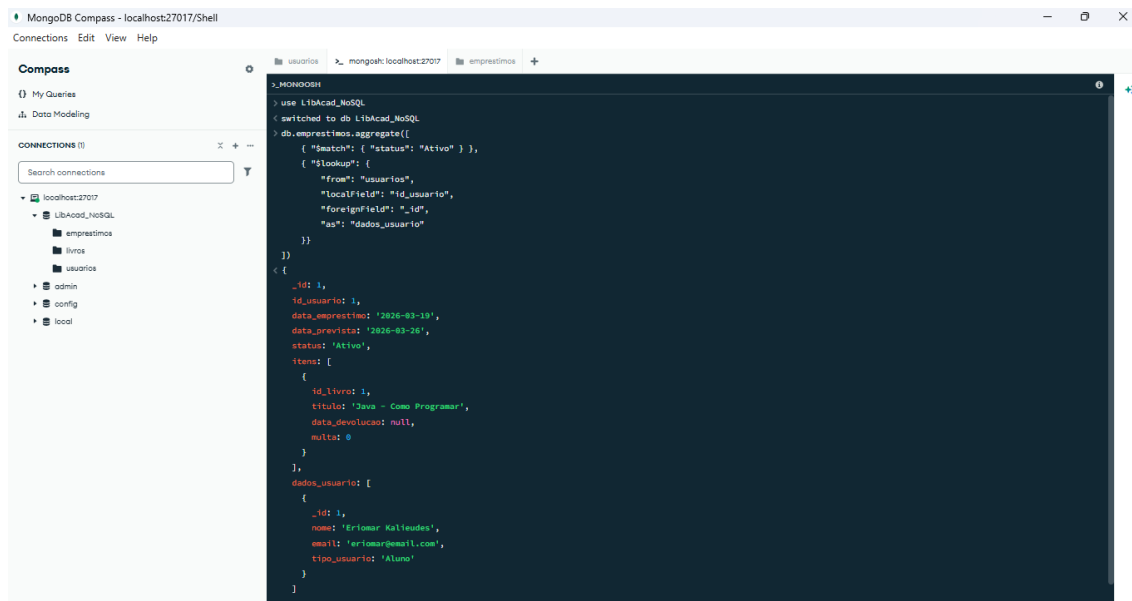


Figura 32 - Agregação de empréstimos com dados do usuário.

A consulta de agregação funcionou perfeitamente, unindo os dados do empréstimo com os dados do usuário responsável. Isso comprova que a modelagem baseada em *Referencing* foi bem-sucedida e que a integridade dos dados foi mantida durante os testes de exclusão (Delete), preservando a consistência dos registros de empréstimos ativos. A consulta de agregação documentada na Figura 32 foi estruturada com o objetivo prático de consolidar as informações de retiradas pendentes com os dados cadastrais completos dos respectivos usuários, simulando com máxima eficiência a operação de junção de tabelas (JOIN) que ocorria no modelo relacional. O pipeline de execução utiliza inicialmente o estágio \$match para realizar um filtro preciso na coleção, selecionando apenas os documentos da coleção emprestimos cujo campo status possua o valor exato de "Ativo". Sequencialmente, o estágio \$lookup é acionado para computar a junção entre a coleção de origem (emprestimos) e a coleção de destino (usuarios). O cruzamento lógico é efetuado mapeando o campo local id_usuario com o identificador único _id da tabela de usuários. Como resultado, as propriedades do usuário associado são embutidas de forma estruturada em uma lista temporária nomeada como dados_usuario, otimizando drasticamente o tempo de resposta em consultas voltadas para relatórios e auditorias administrativas em tempo real.

7 - Comparação entre os dois modelos

Características	Relacional	NoSQL
Estrutura	Tabelas	Documentos
Schema	Rígido	Flexível

Relacionamentos	Joins	Embedding/Referencing
Escalabilidade	Vertical	Horizontal
Integridade	Alta	Flexível
Desempenho em leitura	Médio	Alto
Redundância	Baixa	Alta

Apesar do comparativo mostrar as diferenças entre cada um dos bancos, isso não significa que um deles é melhor do que o outro, mas sim que irá depender do tipo de aplicação em que o banco de dados será usado.

Diante desse comparativo as vantagens que podem ser consideradas para o banco NoSQL são: flexibilidade, onde os documentos não precisam seguir a mesma estrutura exatamente, pois os sistemas mais atuais estão mudando constantemente e com flexibilidade é possível adicionar novos campos, evoluir o sistema mais rápido e adaptar funcionalidades; facilidade de escalabilidade, pois o mongo foi criado pensando na distribuição de dados, sendo possível distribuí-los em diferentes servidores que ajudará a manter o desempenho; redução de joins para usar embedding, onde tudo já estará agrupado dentro do documento, acarretando assim menos consultas, processamento e consultas rápidas.

Além das vantagens também devemos estar cientes das limitações que um banco de dados NoSQL tem, como por exemplo: a maior redundância de dados que evita os joins mas repete as informações; possibilidade de inconsistências devido as várias redundâncias; a necessidade de planejamento da desnormalização, pois é necessário decidir o que embutir e referenciar. No projeto da biblioteca online, os itens dos empréstimos foram embutidos dentro da coleção empréstimos utilizando embedding.

8 - Conclusões sobre vantagens e limitações do NoSQL.

O desenvolvimento da biblioteca online permitiu compreender as diferenças que existem em um banco relacional e o NoSQL. A conversão dos dados do modelo relacional

para o NoSQL mostrou que os documentos oferecem uma maior flexibilidade e melhor desempenho para operações de leitura.

O uso de documentos se mostrou vantajoso pois reduziu bastante a utilização de joins, tornando assim as consultas mais simples e eficientes. Como são flexíveis, adaptam-se facilmente a sistemas atuais, facilitando assim o desenvolvimento da aplicação, além de permitir agrupar várias informações em um único documento.

Desta forma, o presente projeto cumpre integralmente os requisitos estabelecidos para a unidade, englobando a análise do modelo relacional original, a reestruturação fundamentada para o MongoDB (utilizando padrões de *Embedding* e *Referencing*), a execução dos scripts de migração e a validação do banco através de operações CRUD e consultas de agregação.

Por fim, de acordo com o desenvolvimento do presente projeto, conclui-se que a escolha entre um banco de dados relacional e NoSQL, sempre dependerá das necessidades do sistema considerando fatores de escalabilidade, desempenho, consistência e flexibilidade.

9 - Controle de Versão (GitHub)

Atendendo aos requisitos do edital para o gerenciamento do código-fonte e facilitação do trabalho em equipe, o desenvolvimento deste projeto foi integralmente versionado utilizando o sistema Git.

Todos os artefatos técnicos, incluindo os scripts de extração SQL, os scripts de migração NoSQL (mongosh) e os arquivos finais .json com as exportações das coleções, encontram-se hospedados em um repositório público no GitHub.

Link de acesso ao repositório: <https://github.com/thalesfrts/Projeto-NoSQL-LibAcad>

A Figura abaixo apresenta o histórico de commits do repositório, evidenciando o uso contínuo do controle de versão durante as etapas do projeto.

Commits · thalesfrts/Projeto-NoSQL-LibAcad/commits/main/

<> Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

Commits

main

All usersAll time

Commits on Jun 9, 2026

Add files via upload

thalesfrts authored now

Verified32acd95

Add files via upload

thalesfrts authored 1 minute ago

Verifiedfd39680

Add files via upload

thalesfrts authored 1 minute ago

Verified8c5a559

Add files via upload

thalesfrts authored 2 minutes ago

Verifiedde81654

Create README.md

thalesfrts authored 3 minutes ago

Verifieddba1c58